

Python Interview Q&A Bank for Applied AI Engineering Roles

Q1. What are the main differences between a list and a tuple in Python?

Answer:

Lists are mutable, meaning they can be modified after creation, while tuples are immutable. Lists use square brackets `[]` and tuples use parentheses `()`. Tuples are generally faster and more memory efficient. Tuples are commonly used for fixed configurations or returning multiple values.

Q2. Explain the difference between `==` and `is` in Python.

Answer:

`==` checks whether two objects have the same value, while `is` checks whether both variables refer to the same object in memory. Example: two separate lists with the same values may return `True` with `==` but `False` with `is`.

Q3. What are Python decorators?

Answer:

Decorators are functions that modify the behavior of another function without changing its code. They are commonly used for logging, authentication, caching, retry mechanisms, and performance monitoring.

Q4. What is the purpose of `*args` and `**kwargs`?

Answer:

`*args` allows a function to accept a variable number of positional arguments, while `**kwargs` accepts variable keyword arguments. These are heavily used in reusable APIs and framework development.

Q5. What are Python generators and why are they useful?

Answer:

Generators produce values lazily using the `yield` keyword instead of returning all values at once. They are memory efficient and useful when processing large datasets, streaming data, or AI pipelines.

Q6. Explain list comprehension with an example.

Answer:

List comprehension provides a concise way to create lists. Example: `squares = [x*x for x in range(5)]` creates a list of squared numbers. It improves readability and performance.

Q7. What is the difference between deep copy and shallow copy?

Answer:

A shallow copy copies references of nested objects, while a deep copy recursively copies all nested objects. Deep copy prevents unintended modifications in complex AI data structures.

Q8. What is exception handling in Python?

Answer:

Exception handling allows programs to handle runtime errors gracefully using try, except, finally, and raise blocks. It is essential for robust APIs, AI pipelines, and automation workflows.

Q9. What are Python virtual environments?

Answer:

Virtual environments isolate project dependencies so different projects can use different package versions. Tools like venv and conda are commonly used in AI development.

Q10. Explain the difference between multithreading and multiprocessing.

Answer:

Multithreading runs multiple threads within the same process and is useful for I/O-bound tasks. Multiprocessing uses separate processes and is better for CPU-intensive operations like ML preprocessing.

Q11. What is the Global Interpreter Lock (GIL)?

Answer:

The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time. It affects CPU-bound multithreaded applications but not multiprocessing.

Q12. What are Python context managers?

Answer:

Context managers manage resources automatically using the with statement. They ensure files, database connections, or network resources are properly closed.

Q13. How does Python manage memory?

Answer:

Python uses reference counting and garbage collection for memory management. Unused objects are automatically cleaned to optimize memory usage.

Q14. What is the difference between `append()` and `extend()`?

Answer:

`append()` adds a single element to a list, while `extend()` adds multiple elements from another iterable.

Q15. What are lambda functions?

Answer:

Lambda functions are small anonymous functions written in a single line. They are commonly used with `map()`, `filter()`, and sorting operations.

Q16. Explain Python's `map()`, `filter()`, and `reduce()` functions.

Answer:

`map()` transforms elements, `filter()` selects elements based on conditions, and `reduce()` accumulates results into a single value. These functions support functional programming patterns.

Q17. What is the difference between a Python module and a package?

Answer:

A module is a single Python file, while a package is a directory containing multiple modules and an `__init__.py` file.

Q18. Why are dictionaries important in AI engineering?

Answer:

Dictionaries provide fast key-value lookups and are commonly used for JSON handling, configuration management, embeddings metadata, and caching.

Q19. What are Python dataclasses?

Answer:

Dataclasses reduce boilerplate code for classes by automatically generating methods like `__init__`, `__repr__`, and `__eq__`.

Q20. Explain inheritance in Python.

Answer:

Inheritance allows one class to acquire properties and methods from another class. It supports code reuse and modular software design.

Q21. What is polymorphism in Python?

Answer:

Polymorphism allows the same method name to behave differently depending on the object or class using it.

Q22. What is the difference between synchronous and asynchronous programming?

Answer:

Synchronous programming executes tasks sequentially, while asynchronous programming allows tasks to run concurrently using `async` and `await`.

Q23. What is asyncio in Python?

Answer:

`asyncio` is a Python library used for asynchronous programming. It is useful for APIs, concurrent requests, and scalable AI services.

Q24. What are REST APIs and how are they used in Python?

Answer:

REST APIs allow communication between systems over HTTP. Python frameworks like `Flask` and `FastAPI` are commonly used to build AI inference services and backend APIs.

Q25. What is JSON serialization in Python?

Answer:

JSON serialization converts Python objects into JSON format for data exchange between APIs, frontend systems, and AI services.

Q26. How would you optimize Python code performance?

Answer:

Performance optimization techniques include using generators, vectorized operations with `NumPy`, caching, profiling, multiprocessing, and reducing unnecessary loops.

Q27. What are Python type hints?

Answer:

Type hints specify expected data types for variables and functions. They improve code readability, IDE support, and maintainability in large AI systems.

Q28. What is logging and why is it important?

Answer:

Logging helps track application behavior, debug issues, monitor AI workflows, and analyze failures in production systems.

Q29. How does FastAPI help in AI engineering?

Answer:

FastAPI enables high-performance API development with automatic validation, asynchronous support, and Swagger documentation. It is widely used for deploying AI and ML models.

Q30. Explain a practical Python workflow in an Applied AI project.

Answer:

A typical workflow includes reading data, preprocessing inputs, calling ML models, handling APIs, logging outputs, storing results, and managing exceptions. Python integrates all these components efficiently.